

作业二：Google 三驾马车的问题发现与设计决策

本文围绕 MovieLens 数据治理 Agent，形成“工作负载与场景 → 问题及证据 → 设计决策 → 所需能力 → 逻辑架构 → 分阶段演进”的推理链。

1.1 问题发现

P1：节点故障可能使原始输入或正式产物不可用，并迫使治理任务重算

项目	内容
现象或风险	原始 MovieLens 文件、候选清洗结果和正式报告若位于故障节点，可能出现“任务记录仍在、实际文件不可读”或“报告可见、对应指标文件已丢失”的不一致状态。
触发条件	原始文件、清洗数据、质量指标和报告分散到多个节点后，发生磁盘/进程/节点失效或任务执行中节点失联；当前尚无实际故障率与恢复时间数据。
影响对象	原始输入、清洗任务、正式结果、Agent、用户和运维人员。
影响结果	原始文件丢失会阻断后续清洗；候选或清洗结果丢失会迫使整任务重跑；正式报告或指标丢失会使已返回的结论无法复核；若所有对象同等冗余，又会增加存储成本。
判断证据	GFS 将机器故障视为常态，并通过 Chunk 副本、心跳和恢复维持可用性；实验要求基于真实 Hadoop 计算返回结果。MovieLens 当前规模下的实际故障频率仍需实测。
优先级	高：当前至少要界定“哪些对象必须保住、哪些对象允许重算”，否则节点故障会直接影响真实结果与可追溯性；具体副本数和恢复参数以后根据规模与故障数据确定。

核心问题：原始文件、可重跑的候选/清洗结果和正式报告的恢复要求不同，系统如何避免故障对象被误当作可用正式结果？

关联机制（GFS）：Chunk、多副本、心跳、操作日志、故障检测与恢复。

关联决策：→ D1。

P2：多次治理任务可能使原始数据、规则、指标和报告发生版本错配

项目	内容
现象或风险	同一 MovieLens 输入被不同清洗规则或参数处理后，若仍使用固定文件名或“最新结果”引用，清洗后的 ratings/movies/users、清洗前后多维指标和报告可能来自不同任务。
触发条件	用户多次提交相似请求、修改清洗规则、更新输入数据，或多个任务并发处理同一数据集；当前实验任务数量较少，但不能依赖“只有一次运行”的隐含假设。
影响对象	清洗数据、多维指标、报告、Agent 和用户。

项目	内容
影响结果	报告可能引用错误规则或错误数据版本的评分；后续任务可能覆盖旧结果；清洗前后指标无法成对比较；历史结论无法复现。
判断证据	实验要求返回清洗前后多维评分及解释，天然要求同一治理链路内的产物可对应；Bigtable 的时间戳多版本和行键设计说明覆盖写不适合保留可追溯历史。并发和多版本是后续扩展场景，需在真实工作负载中验证频率。
优先级	高：当前必须保证一次治理内的产物不互相错配；复杂的版本保留数量和自动清理策略可在历史任务增长后再确定。

核心问题：系统如何让一份报告只能引用本次任务采用的原始数据、规则、清洗结果和指标，而不是任意“最新”文件？

关联机制（Bigtable）：RowKey、时间戳多版本、列族、数据模型设计。

关联决策：→ D2。

P3：将大文件、任务状态和结果索引混合组织，可能使查询、清理和一致性校验失效

项目	内容
现象或风险	若任务状态、版本、产物位置与原始/清洗文件混在同一目录或仅靠文件名推断，按任务查询状态、按数据集读取正式报告、比较历史指标或清理过期结果都可能退化为目录遍历，并产生“索引存在但文件缺失”或反向脱节。
触发条件	历史任务、版本和报告数量增长；Agent/用户需要按任务标识查询状态、按数据集/版本读取结果；当前任务数少，尚无查询延迟基线。
影响对象	Agent、用户、运维人员、任务元数据和实际数据文件。
影响结果	查询延迟与扫描范围一同增长；错误清理可能删除仍被报告引用的文件；状态或版本信息与实际结果不一致时，Agent 可能读取错误对象。
判断证据	GFS 把文件元数据与 Chunk 数据分离，Bigtable 以 RowKey/Tablet 支撑按键访问和版本管理；实验要求 Agent 查询任务状态并按数据集/任务取得结果。具体访问频率和索引规模仍需压测确认。
优先级	高：当前需要先确定元数据与实际数据的职责边界及最小查询键；是否需要复杂索引、分区或缓存留待任务和查询量增长后评估。

核心问题：系统如何用任务、数据集和版本定位实际产物，并在文件丢失、重跑或清理后仍能识别其真实状态？

关联机制：GFS（Master 元数据）、Bigtable（RowKey、Tablet、SSTable、Compaction、Bloom Filter）。

关联决策：→ D1/D2。

P4：任务切分、Shuffle 与负载不均可能形成长尾，导致治理任务延迟

项目	内容
现象或风险	清洗和五维评分需要批量扫描、转换与聚合数据。切分过粗会限制并行度，切分过细会增加调度开销；热点键、Shuffle 数据增长或个别慢任务又会让少数任务决定整个作业的完成时间。
触发条件	数据量与切分粒度不匹配；按 movielid、userId 或异常类型聚合时少数键关联的记录明显更多；一条输入产生多组中间键值且缺少局部聚合；磁盘异常、复杂记录或多个任务争用 CPU、内存、磁盘和网络。
影响对象	数据处理任务、Agent、用户、运维人员。
影响结果	完成时间不可预测、超时、资源成本增加；前端可能把仍在运行的任务误判为失败并重复提交。
判断证据	实验工作负载（批量清洗和五维评分）、MapReduce 的任务粒度、Shuffle、分区、数据本地性和备份任务机制 [6]，以及长尾延迟研究 [7]。热点与并发场景属于工作负载推演，仍需实测验证。
优先级	高：现在规定切分、观测和调节能力；具体分片大小、分区函数和资源参数以后根据运行数据确定。

核心问题：系统如何在数据规模和分布变化时合理拆分工作、识别倾斜与慢任务，避免少数分片拖慢整个治理作业？

关联机制（MapReduce）：任务划分、任务粒度、Shuffle、分区、数据本地性、备份任务。

关联决策：→ D3。

P5：失败重试和重复提交可能产生冲突输出或错误发布

项目	内容
现象或风险	节点失联、网络中断或等待超时不一定表示原执行已经停止；若 Agent、用户或计算框架再次启动同一逻辑任务，多个执行尝试可能并存并相互覆盖、重复追加或混合输出。
触发条件	节点、进程、协调者或网络故障；启用超时/失败自动重试；用户重复点击或客户端重发；计算框架重新执行失败任务或启动备份任务。多个尝试若可写入同一正式结果且缺少去重、隔离和单一发布机制，就会形成冲突。
影响对象	任务状态、清洗数据、质量指标、报告、Agent、用户、运维人员。
影响结果	重复记录、指标重复计数、部分结果、正式结果被覆盖、状态与输出不一致、资源浪费。
判断证据	MapReduce 把重新执行作为容错手段，并通过确定性处理、临时输出和原子提交约束重复执行 [6]；安全重试和唯一请求标识的工程实践进一步说明业务层仍需幂等控制 [8]。
优先级	高，但低于先跑通正常治理链路。当前先保证失败输出不可冒充正式结果、不同任务可区分；自动重试、退避和阶段级恢复后续增强。

核心问题：一个逻辑任务存在多个执行尝试时，系统如何确定任务终态并保证只有一份正确结果正式生效？

关联机制（MapReduce）：失败检测、任务重新执行、备份任务、确定性语义、临时输出与原子提交。

关联决策：→ D3/D4。

P6：Agent 返回的状态、指标和解释可能与真实任务及证据错配

项目	内容
现象或风险	对前端用户而言，Agent 和后端处理过程近似黑盒。若系统没有绑定“本次输入—本次任务—本次规则—本次结果”，Agent 可能引用错误数据集、旧任务或失败尝试的输出，也可能给出无法由计算证据验证的数值和解释。
触发条件	支持多数据集、多版本规则或并发同类任务却缺少明确标识；Agent 在状态与结果更新存在延迟、乱序或缓存时读取信息；失败任务的部分输出仍可读取；结果缺少任务、输入、规则和关键计数等来源信息；Agent 可脱离结构化结果生成状态或数值。
影响对象	Agent、用户、任务与结果数据、审计和运维人员。
影响结果	Agent 编造或误引数值、任务状态不一致、证据不可追溯、错误结果被当成正式结果、问题无法复现。
判断证据	实验明确要求返回“真实结果及相应解释”；MapReduce 使用状态、日志和计数器核查运行与处理量 [6]；数据来源研究和 W3C PROV 强调结果与源数据、处理活动及派生关系的关联 [9][10]。多版本错配属于未来场景推演。
优先级	中等：当前先保证 Agent 数值来自真实计算、失败结果不可作为正式答案；支持多数据集、多规则和并发任务后，再提高完整追溯与审计能力的优先级。

核心问题：系统如何证明 Agent 返回的状态、指标和解释确实对应用户本次输入、实际完成的任务和被采用的正式结果？

关联机制（MapReduce）：状态信息、日志、计数器、重复执行计数去重。

关联决策：→ D4。

问题组合逻辑

P1：数据是否还在、故障后能否恢复	→ 存储可靠性
P2：数据属于哪个版本、产物如何对应	→ 多版本数据组织
P3：如何快速找到数据并管理历史结果	→ 元数据与查询模式
P4：任务如何拆分并避免少数任务拖慢整体	→ 并行计算与长尾治理
P5：失败和重复执行后如何保持唯一结果	→ 故障恢复与幂等发布
P6：Agent 返回内容如何对应真实计算证据	→ 系统可信性与可追溯

- P1 是通用问题的场景化；P2、P3 是数据治理 Agent 更具特色的场景问题。
- 三者分别覆盖存储可靠性、多版本数据组织、元数据与查询模式，可自然导出 D1、D2。

- P4–P6覆盖并行计算、故障重试和系统可信性，导出D3、D4。

1.2 课堂知识映射

GFS

机制 1: Chunk 划分 + 多副本 + 心跳故障检测

1. **解决什么问题**: 将大文件切分为固定大小的 Chunk, 每个 Chunk 保存多份副本到不同节点, Master 通过心跳周期检测节点存活, 任一节点失效后由副本在其他节点恢复可用性, 避免单点故障造成数据永久丢失。
2. **依赖的假设**: 以机器/磁盘故障为常态; 文件以大文件顺序读写为主; 副本带来的额外存储成本可接受; 元数据量远小于数据量。
3. **带来的限制**: 副本带来多倍存储开销; 对"可重新生成"的中间数据也强制副本是浪费; 故障检测存在心跳周期延迟; 副本恢复消耗网络带宽, 可能抢占正常读写; 客户端直连 ChunkServer 使一致性模型弱化(追加写、容忍重复)。
4. **MovieLens 是否类似**: 是。系统当前把原始数据、清洗结果、质量报告都当普通文件落盘, 未区分"原始数据(不可再生)"与"清洗结果(可重跑再生)"的价值差异, 也未规定故障后的恢复边界。依据: 实验要求 Hadoop 处理真实数据并返回可追溯结果; P1 已指出节点故障会造成治理数据不可用、迫使任务整体重算。
5. **形成的能力要求**: 按数据价值分级决定冗余/恢复策略——原始输入和正式报告不能因单节点故障不可用; 候选/清洗结果可在确认其可重跑后降低保护级别。当前只规定这一恢复边界, 不预设副本数、恢复时限或纠删码策略。→ 支撑 D1; 能力: 故障检测、对象级恢复策略、恢复结果校验。

机制 2: 单 Master 元数据集中 + 操作日志 + 快照

1. **解决什么问题**: Master 在内存集中维护"文件→Chunk 映射"、副本位置等元数据, 定位任意文件只需一次元数据查询; 操作日志(WAL)记录每次元数据变更, 配合周期快照, 使 Master 崩溃后能快速重建内存状态。
2. **依赖的假设**: 元数据总量相对数据量很小(内存可容纳); 元数据查询频率远高于实际数据读写; 操作日志落盘顺序追加成本低。
3. **带来的限制**: 单 Master 是单点, 崩溃期间无法定位文件; 内存容量约束元数据规模上限; 元数据与实际数据的一致性需额外机制维护(副本位置会随故障改变); 快照+日志重放存在恢复时间。
4. **MovieLens 是否类似**: 是。随任务和版本累积, 需按"任务编号/数据集/版本"快速定位结果与报告; 若把这种查询需求与大数据文件读写混在一起, 会退化为全量扫描。依据: P3 指出大文件与小型元数据同组织会导致查询低效、元数据与实际文件脱节; 实验要求 Agent 按任务查询状态、按数据集获取结果。
5. **形成的能力要求**: 把"元数据管理"与"数据读写"分离——元数据层记录任务、版本、产物位置和状态, 实际文件保持独立; 元数据失效或文件丢失时必须能够识别不一致。当前不指定元数据产品、缓存或高可用形态。→ 支撑 D1; 能力: 元数据持久化/恢复、按任务和版本定位、元数据—文件一致性校验。

Bigtable

机制 3: 时间戳多版本 + 提交日志

1. **解决什么问题**: Bigtable 为同一行键的每个列族单元格保存多个带时间戳的版本, 读取时可回退到历史版本; 提交日志在写入前先落盘, 保证 Tablet 崩溃后未刷盘的写入可恢复。

2. **依赖的假设**：写入需保留历史而非覆盖；读取通常只关心最新或指定版本；可配置版本保留策略；写入可重放（幂等或至少可恢复）。
3. **带来的限制**：多版本放大存储占用，需 Compaction/垃圾回收清理过期版本；版本保留策略需权衡"可追溯"与"存储成本"；提交日志重放可能引入重复写入，需幂等去重。
4. **MovieLens 是否类似**：是。同一 MovieLens 数据集会被不同规则/参数/时间多次清洗，产生多个清洗版本、评分和报告；若只保留"最新结果"或用固定文件名覆盖，会导致版本混用、结果不可追溯。依据：P2 指出多次治理任务会造成版本混用和产物对应关系不清；实验要求返回清洗前后多维评分和解释，需要能回溯。
5. **形成的能力要求**：为每次治理任务及其产物建立唯一标识和版本；清洗前后指标、清洗结果和报告必须同属一个被采用的任务版本，正式结果不可变。当前可保留全部历史，版本数上限和自动清理策略以后按实际增长确定。→ 支撑 D2；能力：版本标识、血缘关联、不可变发布、历史回溯和过期清理。

机制 4：RowKey 设计 + Tablet 分片 + SSTable/Compaction/Bloom Filter

1. **解决什么问题**：按 RowKey 字典序把数据分布到 Tablet 并切分，使相邻行键查询落在同一 Tablet，避免全表扫描；SSTable 以有序形式持久化数据，Bloom Filter 快速判断行键是否存在，Compaction 合并碎片化 SSTable 并回收过期版本。
2. **依赖的假设**：访问模式以行键点查/范围扫描为主；RowKey 设计决定数据分布与查询性能；写入以追加为主，读取需索引加速。
3. **带来的限制**：RowKey 设计不当会造成热点（如用递增时间戳作前缀导致单 Tablet 过载）；Compaction 在后台消耗 IO，可能影响前台读写；多版本 + 碎片化 SSTable 使读放大，需 Bloom Filter/缓存抵消。
4. **MovieLens 是否类似**：是。系统需按"数据集/任务/版本"查询报告与评分、比较版本、清理过期结果；若数据布局与查询模式不匹配，会产生全表扫描和热点。依据：P3 指出查询模式与数据布局不匹配会导致查询低效和历史结果难管理；实验要求 Agent 按任务和数据集快速获取结果。
5. **形成的能力要求**：设计能支撑"数据集→任务→版本"的定位方式，使报告、多维指标和清洗数据可按同一键关联；读取正式结果不得依赖目录遍历或"最新文件"约定。当前不预设具体行键、分区、索引或缓存实现。→ 支撑 D2；能力：按键查询、范围查询、正式版本过滤和过期清理。

MapReduce

机制 5：任务划分与任务粒度

1. **解决什么问题**：把输入拆成多个 Map 任务、把聚合拆成多个 Reduce 任务，使独立分片能够并行处理，并便于失败后重新调度。
2. **依赖的假设**：输入可按记录或数据块切分；Map 任务之间无强顺序依赖；聚合可按中间键分区；单任务计算量足以覆盖调度开销。
3. **带来的限制**：切分过粗会降低并行度，切分过细会增加调度和中间文件开销。Reduce 数量 R 同时决定输出文件数；R 过小会使单个 Reduce 负载过重，R 过大则增加任务和小文件数量。
4. **MovieLens 是否类似**：是。清洗和质量检查可按记录并行，计数和比例可按键聚合；数据增长后，单任务全量处理会延长等待时间。当前没有压测数据，不能预设最优切分大小。
5. **形成的能力要求**：任务可切分、粒度可调，记录各分片的处理量、耗时和失败情况。→ 支撑 D3；能力：切分、调度、分片观测。

机制 6：Shuffle、分区与数据本地性

1. **解决什么问题**：Shuffle 将相同中间键汇集到对应 Reduce，支持全局计数和按键聚合；数据本地性尽量让 Map 靠近输入数据执行，减少原始数据跨网络传输。

2. **依赖的假设**：聚合可表达为“相同键的数据汇合”；分区函数确定且一致，使相同键始终进入同一 Reduce；计算与数据之间存在可利用的邻近关系。
3. **带来的限制**：Shuffle 会产生网络传输、排序和磁盘溢写；热点键会形成 Reduce 倾斜。自定义分区函数若逻辑不一致会破坏正确性，分区不均则会造成长尾；存算分离环境中，本地性收益可能减弱。
4. **MovieLens 是否类似**：是。五星评分可能汇总缺失、重复、合法值数量或按用户/电影统计分布；少数热门键可能使默认分区负载不均，但需通过真实 Shuffle 指标验证。
5. **形成的能力要求**：记录各阶段输入量、中间数据量和分区耗时，支持局部预聚合、分区调整和倾斜识别。→ 支撑 D3；能力：Shuffle 观测、分区调节、减少无效传输。

机制 7：失败检测、重新执行与确定性语义

1. **解决什么问题**：协调者检测执行单元失败后，把未完成或输出不可访问的任务重新调度，使作业在局部故障后继续推进。
2. **依赖的假设**：任务输入仍可读取，输出具有清晰提交边界，重算成本可接受。若要获得与无故障串行执行等价的强语义，Map 和 Reduce 对相同输入必须是确定性的；非确定性处理仍可运行，但语义会变弱。
3. **带来的限制**：同一逻辑任务可能执行多次；非幂等副作用可能重复发生；频繁失败会造成资源浪费。框架内任务成功不等于业务报告已经安全发布。
4. **MovieLens 是否类似**：是。进程退出、节点失联、前端超时或用户重提都可能产生多个执行尝试；若直接覆盖正式文件，会造成结果冲突。
5. **形成的能力要求**：区分逻辑任务与执行尝试，持久记录状态、失败原因和重试次数；各尝试先写隔离结果，通过校验后再发布。→ 支撑 D3/D4；能力：受控重试、执行隔离、幂等发布。

机制 8：备份任务与慢任务治理

1. **解决什么问题**：作业接近完成时，为剩余未完成任务启动备份执行，主执行或备份执行任一先完成即可结束该任务，降低少数慢任务对整体完成时间的影响。
2. **依赖的假设**：任务可安全重复执行；系统有少量空余资源；最终提交能够处理多个尝试竞争。原论文没有采用复杂的历史速度预测。
3. **带来的限制**：备份执行额外消耗计算和 I/O；若慢任务来自数据倾斜，两个副本可能同样慢；非幂等副作用可能执行两次。
4. **MovieLens 是否类似**：可能存在。热点分区、异常记录处理或资源争用可能形成长尾，但当前没有慢任务数据，不应把推测执行设为默认必选。
5. **形成的能力要求**：先提供任务耗时与进度观测，再决定是否启用备份执行；启用后保证多个尝试只有一个结果能正式生效。→ 支撑 D3/D4；能力：慢任务识别、安全重复执行、单一结果采用。

机制 9：临时输出、原子提交、状态与计数器

1. **解决什么问题**：执行尝试先写临时输出，成功后再提交正式结果；状态和日志用于观察进度、失败与处理量；计数器会剔除同一任务因备份或失败重执行产生的重复计数。
2. **依赖的假设**：候选与正式结果可隔离，提交具有可靠边界，任务和尝试具有唯一标识，核心处理尽量确定。多个文件若要求整体一致，需要额外协调能力。
3. **带来的限制**：单个输出的原子提交不等于多个业务产物可以整体发布，也不证明规则和指标在业务语义上正确。框架计数器只能消除同一作业内的重复尝试，不能识别用户重复提交的两个独立作业。
4. **MovieLens 是否类似**：是。一次治理同时产生清洗数据、质量指标、异常样例和报告，必须避免失败或不完整结果被 Agent 读取，并用关键计数核查处理结果。

5. **形成的能力要求**：论文机制直接支持临时输出、单个 Reduce 输出提交及状态/计数核查；本系统进一步要求候选结果隔离、校验后幂等发布，并约束 Agent 只使用正式结构化结果。→ 支撑 D4；能力：结果隔离、校验、追溯和可信读取。

1.3 工业界演进调研：多副本 → 纠删码 / 分层存储

1. 经典方案原本解决什么问题

GFS 用**多副本**解决节点故障下的数据可靠性：把每个 Chunk 存成多份副本分布到不同节点，任一节点失效后由剩余副本继续服务、并触发重副本。副本方案简单、恢复快、延迟低，代价是固定多倍存储开销——HDFS 默认 3 副本带来 **200% 的额外存储开销**（3x 空间）[1]。

2. 工作负载发生了什么变化

副本方案在数据量爆炸后暴露出两个问题：

- **存储成本过高**：3x 副本在 PB 级规模下意味着三倍磁盘、机房占地和功耗，成为最主要的成本来源。微软 Windows Azure Storage 的实证显示，纠删码可**降低存储成本超过 50%**，机房占地和功耗约减半 [2]。
- **冷热不均**：大数据集群里大部分数据是“写一次、偶尔读”的冷数据，只有少数正在被处理的热数据需要副本那样的低延迟随机访问。对所有数据一律 3 副本，等于为大量冷数据支付了它根本用不上的高频访问能力。

3. 新方案解决了哪些旧问题，又引入哪些新问题

3a. 纠删码（Erasure Coding, EC）

解决旧问题：用 K 个数据块 + M 个校验块的方式，把存储开销从副本的 3x 压到约 1.5x 甚至更低。以 HDFS 默认策略 **RS-6-3-1024k** 为例：6 数据块 + 3 校验块，一个 6 块的文件三副本要 18 块，EC 只需 9 块 [1]。

引入新问题：

- **恢复读放大**：副本坏一块只需读 1 块复制；而 EC 恢复任意 1 块都必须读满 K 块（RS(6,3) 要读 6 块）做解码运算，网络和 CPU 开销显著 [2]。Facebook 的实证数据更直观：RS 编码只覆盖约 8% 的数据，却贡献了约 20% 的网络修复流量 [3]。
- **牺牲写入灵活性**：EC 文件不支持 append / truncate / hflush / hsync 等随机写操作 [1]，即“空间换不了即时随机写”，只适合不可变数据。
- **算法演进**：Reed-Solomon（RS）是最通用的 MDS 码，但恢复读放大高；XOR（如 XOR-2-1-1024k）最省 CPU 但只能容 1 块故障 [1]。工业界为此提出 **LRC（局部恢复码）**，把校验块拆成“局部校验 + 全局校验”，微软 LRC(12,2,2) 把单块恢复读放大从 RS 的 12 块降到 6 块，存储开销仅 1.33x [2]。

3b. 分层存储（Tiered / Hot-Warm-Cold）

解决旧问题：按数据冷热把不同数据放到不同成本和性能的介质上。HDFS 官方把存储分为 Hot（DISK）、Warm（DISK + ARCHIVE）、Cold（ARCHIVE）三档，用 Mover/SPS 工具按策略在 SSD/HDD/归档盘之间迁移 [4]。云对象存储把同一思想推向极致：AWS S3 的 Glacier 冷层用“极低存储单价 + 按 GB 取回费 + 长取回延迟”定价，如 Glacier Deep Archive 首字节延迟达 12–48 小时 [5]。

引入新问题：

- **冷热判断与迁移成本**：需要访问频率、数据年龄等指标判断冷热，误判的代价不只是慢，还可能是取回费；迁移本身消耗 IO 和带宽。
- **取回延迟**：冷数据被用户突然访问时，延迟从毫秒级劣化到分钟甚至小时级 [5]。

4. 哪些结论适用于本系统，哪些仅适用更大规模

适用的结论（可作为设计方向，不必现在落地）：

- "热数据用副本、冷数据用 EC/归档"的分级思想，与 P1 提出的"原始数据不可再生需高保障、清洗结果可重跑可降低保障"是同一逻辑，可直接支撑 D1 的分级保护决策。
- "先写副本、数据封存（不可变）后再后台转 EC"的工程模式 [2]，与 P2 的"正式结果不可变、保留历史版本"方向一致——只有不可变的历史结果才具备转 EC 的条件。

仅适用更大规模的结论（本系统当前规模不应采纳）：

- EC 的 >50% 成本节省，只有在数据量大到 3x 副本的存储成本开始超过恢复时的 CPU/网络代价时才划算。MovieLens 是课程实验规模，数据量远未到 PB 级，副本的 3 倍开销完全可接受，反而换来恢复简单、延迟低、支持追加写。
- 分层存储（SSD/HDD/归档、S3 Glacier 取回模型）面向的是海量冷数据；本系统的历史结果数量小，冷热分层和取回费的收益覆盖不了其判断与迁移复杂度。

规模触发点的判据（供未来重评 D1 时参考）：业界用"数据封存 1GB / 30–180 天未访问"这类客观指标判断数据是否变冷 [2][5]；当 MovieLens 数据量增长到副本开销成为主要成本、且存在大量长期不再访问的历史结果时，再引入 EC 和分层存储。

参考资料

[1] Apache Hadoop 官方文档，HDFS Erasure Coding, <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HDFSErasureCoding.html>（访问于 2026-10）

[2] Cheng Huang et al., *Erasure Coding in Windows Azure Storage*, USENIX ATC 2012 (Best Paper), https://www.usenix.org/system/files/conference/atc12/atc12-final181_0.pdf

[3] *Optimal Locally Repairable Codes*, arXiv:1301.7693, <https://ar5iv.labs.arxiv.org/html/1301.7693>（转引 Facebook HDFS-RAID 的修复流量数据）

[4] Apache Hadoop 官方文档，Archival Storage（异构存储与归档），<https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/ArchivalStorage.html>（访问于 2026-10）

[5] AWS S3 Storage Classes 官方页, <https://aws.amazon.com/s3/storage-classes/>（访问于 2026-10）

[6] Jeffrey Dean, Sanjay Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*, OSDI 2004, <https://research.google.com/archive/mapreduce-osdi04.pdf>

[7] Jeffrey Dean, Luiz André Barroso, *The Tail at Scale*, Communications of the ACM 56(2), 2013, <https://research.google/pubs/the-tail-at-scale/>

[8] Malcolm Featonby, *Making retries safe with idempotent APIs*, Amazon Builders' Library, <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

[9] W3C, *PROV Overview: An Overview of the PROV Family of Documents*, W3C Working Group Note, <https://www.w3.org/TR/prov-overview/>

[10] Peter Buneman, Sanjeev Khanna, Wang-Chiew Tan, *Why and Where: A Characterization of Data Provenance*, ICDT 2001, https://www.research.ed.ac.uk/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf

1.4 设计决策

D1: 元数据与数据读写分离, 原始数据与派生结果分级保护

- **关联问题:** P1、P3
- **设计决策:** 把系统对象明确分为实际数据和逻辑元数据。实际数据包括 MovieLens 原始文件、隔离异常记录、候选/正式清洗后的 ratings/movies/users、质量指标和报告; 逻辑元数据包括数据集、规则、数据/结果版本、产物位置及其血缘。元数据不能代替实际文件, 也不能仅由目录名或“最新文件”推断。原始输入必须在单节点故障后仍可获得; 可从确定输入和规则重跑的候选/清洗结果允许重算恢复; 正式报告及其证据按正式产物保留。
- **所需能力:** 实际数据与元数据的独立持久化边界; 由数据集、任务和版本定位实际产物; 元数据—文件存在性与版本一致性校验; 按对象可重算性配置恢复级别; 故障后识别“仍可读、待重算、缺失”的状态。
- **约束:** 课程实验先满足单节点故障后不把缺失对象当作有效结果, 不预设副本数、恢复时限、可用性指标或纠删码策略; 任务状态和执行尝试的细粒度记录由 D3 负责, 候选结果能否成为正式结果由 D4 负责。
- **依据:** GFS 将文件元数据与 Chunk 数据分离, 并用日志恢复元数据状态; 1.3 表明副本、纠删码和分层存储须按对象冷热及恢复代价取舍; P1 要求区分不可再生原始数据与可重跑派生数据, P3 要求避免用扫描实际文件代替查询元数据。
- **代价与保留风险:** 元数据自身仍可能故障或滞后, 元数据存在不等于文件一定可读; 分级错误会使重跑代价高于预期; 一致性校验会增加写入、恢复和清理步骤。当前不要求高可用元数据集群或自动冷热迁移。
- **验收条件:** 模拟一个实际数据对象不可读时, 系统可识别其类别, 原始输入不会被静默替换为缺失状态, 可重跑对象不会被当作正式可读结果; 按数据集/任务/版本可定位对应文件而无需遍历全部目录; 元数据指向缺失或版本不符文件时返回不一致而非正常结果。
- **触发重评的条件:** 原始数据、正式报告或证据的容量与恢复成本显著上升; 副本成本成为主要开销; 大量历史对象长期不访问; 元数据查询成为可观测瓶颈; 或出现无法靠重跑恢复的派生结果。

D2: 治理结果按“数据集→任务→版本”唯一标识, 正式结果不可变并保留历史

- **关联问题:** P2、P3
- **设计决策:** 以“数据集标识 / 逻辑任务标识 / 数据与规则版本 / 正式结果版本”关联治理产物。一次被采用的治理结果必须形成完整血缘: 原始文件 → 清洗后的 ratings/movies/users → 清洗前后多维指标 → 报告; 报告只可读取同一正式版本内的指标和证据。正式版本发布后不可静默覆盖, 合法的新治理形成新的版本或发布记录。D2只定义数据语义和版本关系; D3记录同一逻辑任务的执行尝试, D4决定哪一个尝试可发布为该正式版本。
- **所需能力:** 数据集、规则、任务和正式结果的唯一标识; 清洗前后指标、异常样例、清洗数据和报告的成组关联; 版本血缘查询; 正式版本过滤; 不可变记录; 历史版本保留与按策略清理。
- **约束:** 实验必须能比较同一治理链路的清洗前后多维评分, ratings/movies/users 三类清洗产物不得跨任务混合; 当前任务少, 可先全量保留历史; 不预设版本编号格式、版本数量上限、自动删除周期或具体索引产品。

- **依据：**Bigtable 的 RowKey 与时间戳多版本表明访问键和版本语义应显式保存，而非靠覆盖写推断；P2 指出规则/数据/报告错配会使结果不可复现，P3 指出历史结果需要可定位查询；1.3 的封存后转 EC 模式也依赖结果已成为不可变历史版本。
- **代价与保留风险：**版本和血缘会增加元数据及存储量；标识设计不当会误把不同意图合并或把同一结果拆散；保留策略过松会积累历史对象，过严会破坏复现。版本关系本身不能阻止失败尝试发布，仍依赖 D4 的候选隔离和校验。
- **验收条件：**同一输入以不同规则运行时，系统可同时定位并区分各自正式结果；任一报告可反查唯一的原始输入版本、规则版本、清洗数据及前后指标；读取旧报告不会因新任务完成而改变其引用内容；查询“最新正式结果”不会返回候选、失败或未发布版本。
- **触发重评的条件：**规则或输入版本频繁变化；历史版本、用户查询或审计需求增长；需要自动过期清理、跨数据集比较或支持增量治理；或现有版本键无法表达新的数据集/规则关系。

D3：治理计算采用可切分、可观测、可恢复的异步任务模型

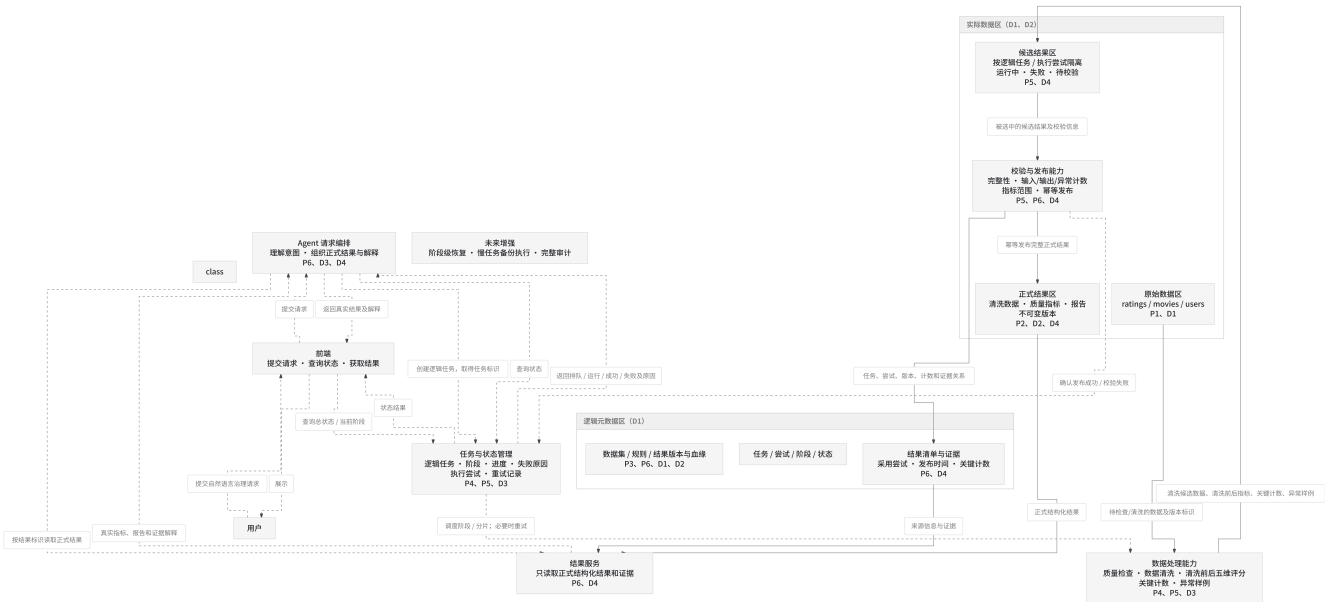
- **关联问题：**P4、P5
- **设计决策：**前端或 Agent 提交请求后创建具有唯一标识的逻辑任务，不让一次同步调用持续到全部计算结束。任务可拆成处理阶段或分片；系统记录每次执行尝试及其状态。当前允许失败后整任务重跑，只有重跑成本超过可接受范围时才升级为阶段级或分片级恢复。
- **所需能力：**逻辑任务标识和执行尝试标识；“已接收/排队、运行中、成功、失败”的总状态及合法转换；运行中显示数据准备、清洗前检查与评分、清洗、清洗后检查与评分、结果生成/发布等当前阶段；分片处理量、耗时、失败原因和重试次数可观测；切分与分区策略可调；超时、可重试故障类型、重试上限和退避可配置；前端可提交任务、查询状态并在成功后获取结果。
- **约束：**实验首先跑通“请求→清洗与评分→真实结果”的基础链路，自动重试和阶段恢复后续增强；当前缺少真实耗时、失败率和服务目标，不预设固定超时、重试次数、并发数或切分大小；任何重试不能绕过 D4 的候选结果隔离和正式发布边界。
- **依据：**MapReduce 按输入分片调度任务，记录任务状态，重新执行失败任务，并提供状态、处理字节和失败信息 [6]；MovieLens 治理由交互请求触发但耗时随数据量和资源状态变化，因此需要将前端交互与长任务生命周期分离。
- **代价与保留风险：**需要增加任务状态持久化、接口、状态清理、监控和故障测试；阶段越细，状态转换和测试组合越多。暂时失联可能被误判为失败并产生并发重试，迟到消息可能覆盖终态，错误重试策略可能形成重试风暴；阶段恢复还需防止读取不完整或版本不兼容的检查点。
- **验收条件：**提交后可按唯一任务标识查询总状态和当前阶段；状态按规则转换，旧尝试的迟到消息不能覆盖终态；可重试故障按策略处理，确定性错误不会无限重试；达到超时或重试上限后明确失败；同一输入和规则重试后的采用结果与无故障执行一致；慢阶段或慢分片可定位。未启用自动重试时，至少保证失败可识别、未完成结果不可作为成功结果返回。
- **触发重评的条件：**任务耗时超过用户可接受范围；整任务重跑成本超过保存阶段结果的成本；任务持续排队或资源失衡；重试成功率过低或超时误判频繁；工作负载转为持续增量/低延迟；规则或数据版本频繁变化导致阶段结果难以复用。

D4：结果采用“隔离生成—证据校验—幂等发布”的可信发布模型

- **关联问题：**P5、P6
- **设计决策：**每个执行尝试在按“逻辑任务标识/执行尝试标识”隔离的候选空间中生成输出，运行中、失败或未通过校验的候选结果不对结果服务和 Agent 可见。候选结果完整且业务校验通过后，以重复调用也只生效一次的幂等动作发布为正式结果。Agent 只能依据正式结构化结果生成数值和解释；正式结果不得被无记录覆盖，合法的新一轮处理形成新版本或发布记录。

- **所需能力：** 候选与正式结果的命名和访问边界；发布幂等标识和并发冲突控制；必要产物齐全性及输入数、输出数、异常数、指标范围校验；结果清单记录任务、被采用的尝试、发布时间、状态和关键计数；多数据集、多规则阶段再完整记录输入版本、规则版本和证据；结果服务只暴露正式结果，Agent在没有正式结果时只能返回运行中、失败或证据不足。
- **约束：** 基础阶段不要求完整版本平台和审计系统，但必须保证失败输出不可见、同一次发布不重复生效、Agent数值来自真实正式结果；多版本、长期审计和人工确认以后增强。本次不指定目录、表、文件系统或事务产品，也不写死幂等键格式、版本编号和候选结果保留时间。
- **依据：** MapReduce先写私有临时文件、忽略已完成任务的重复完成消息，并通过原子重命名使重复Reduce尝试只留下一个最终输出，同时要求额外副作用具备原子性和幂等性 [6]；安全重试实践说明唯一请求标识可区分重复请求与新意图 [8]；P6进一步要求任务、输入、规则和结果可追溯 [9][10]。
- **代价与保留风险：** 需要维护候选结果、结果清单、发布状态和清理策略；发布校验增加步骤与等待，多版本和证据增加存储与查询复杂度，多产物整体发布需要额外协调。幂等标识定义不当可能误合并新请求或漏掉重复请求；并发发布可能选错尝试；清理过早会丢失排障证据，过晚会积累无效数据；完整性校验仍不能发现全部规则逻辑错误。
- **验收条件：** 每个候选结果可追溯到唯一任务和尝试；失败、运行中或未通过校验的候选结果不可读取；同一成功尝试重复发布只产生一个正式结果，合法新任务不会被错误合并；多个尝试竞争时只有一个被采用，迟到尝试不能覆盖正式结果；发布中断后可安全重试且不形成半发布状态；Agent在任务未成功或证据不足时不返回伪造分数；新一轮合法处理不静默覆盖旧结果。
- **触发重评的条件：** 开始支持多个上传数据集、数据/规则多版本或大量历史查询；需要跨多个异构系统整体发布；并发发布冲突显著增加；结果需要合规、长期审计或人工确认；候选结果和证据的存储清理成本明显上升；引入增量或流式结果。

1.5 逻辑架构图



图例

图形/箭头	含义
实线框	当前实验必须具备的逻辑角色或能力
虚线框	本次作业提出、后续实现的设计要求

图形/箭头	含义
浅色框	达到触发条件后再引入的未来能力
实线箭头	数据流：原始数据、候选结果、正式结果和证据
虚线箭头	控制流：提交、调度、查询、重试和发布确认

图中关键边界

- **逻辑任务与执行尝试分开**：一次逻辑任务可以产生多个执行尝试，但最终只采用一个成功尝试。
- **候选结果与正式结果分开**：运行中、失败或未通过校验的结果不能被 Agent 或结果服务读取。
- **任务控制与实际数据分开**：任务与状态管理记录“做什么、做到哪里、是否成功”；原始数据、清洗数据和报告属于实际数据区。
- **计算事实与 Agent 解释分开**：数据处理能力产生结构化指标、计数和证据，Agent 只能基于结果服务返回的正式结果组织解释。
- **任务成功与结果发布分开**：执行尝试完成不等于结果正式可见，必须经过完整性校验和幂等发布。